

Exercise 1: Tokenization and Subword Vocabulary Analysis

Learning Objectives

At the end of this laboratory exercise, students should be able to:

- Apply basic text normalization techniques to prepare a corpus for tokenization
- Describe algorithms of selected subword tokenization methods.
- Apply multiple subword tokenization algorithms on Cebuano dataset.
- Compare the tokenization outputs produced by different subword algorithms on Cebuano words.

Software Requirements

Python 3.x

Jupyter Notebook or Google Colab

Instructions

Task 1. Text Normalization

Perform appropriate preprocessing techniques to clean the corpus. The preprocessing may include:

- converting text to lowercase;
- removing extra whitespaces;
- removing punctuation (when appropriate);
- removing non-text symbols;
- removing digits (if justified);
- applying Unicode normalization (if necessary).

Document every preprocessing step performed and justify why it was applied.

Task 2. Vocabulary Creation

After preprocessing, tokenize the corpus into words; construct the vocabulary containing all unique words; compute:

- total number of word tokens;
- total number of unique word types (vocabulary size)

Task 3. Content Word Analysis

Identify the content words in the corpus.

Extract the following words:

- Top 10 most frequent content words
- Bottom 10 least frequent content words

For each word, report the relative frequency. Present the results in tabular form.

Task 4. Research

Select three (3) subword tokenization methods and research about them. For each algorithm, discuss the main idea tokenization procedure, and limitations. Include also other information such as the developer(s) and in what models these were implemented. Support the discussion with appropriate references.

Task 5. Subword Tokenization

Using available Python libraries, tokenize the corpus using each selected algorithm. Some popular libraries include the Huggingface Tokenizers, SpaCy, NLTK, and SentencePiece. After tokenizing the corpus, generate the token vocabulary and determine the corpus’ total token count and total token types.

Task 6. Comparative Analysis

Select 3 words from the corpus that represent rare words (few instances in the corpus) and morphologically complex or long words. Compare how each tokenizer segments these words and

discuss similarities and differences in segmentation. How did the tokenizer handle affixes in your sample words?

Expected Output

Submit a report containing the outputs for each task, together with the Python notebook used to conduct the experiments in UVEC.